

INTERVIEW PREPARATION GUIDE

Auto EDA

Automated Exploratory Data Analysis — Full-Stack Project Deep Dive



FastAPI + Python



React 18 + Vite



Pandas + PySpark



Azure · AWS · GCP · Databricks



Docker + nginx



Plotly.js Charts



REST API with Auth



SQLAlchemy + PostgreSQL

github.com/ChaudharyPratham/auto_eda

Full-Stack · Data Engineering · Cloud · REST API

May 2026

CONTENTS

What's Inside This Guide

Everything you need to confidently explain Auto EDA to an interviewer.

1

What is Auto EDA? (Elevator Pitch)

One-paragraph explanation + the problem it solves

2

Features Deep Dive

Every feature explained — what it does and how it works under the hood

3

Architecture & Tech Stack

System design, data flow, why each technology was chosen

4

API Design & Security

REST endpoints, API key auth, CORS, SHA-256 hashing

5

Interview Questions & Answers

30+ likely follow-up questions with clear, honest answers in plain English

6

Challenges & How I Solved Them

Real problems faced during development — great for behavioral questions

7

Resume Bullet Points

Copy-ready impact statements for your CV / resume

SECTION 1

What is Auto EDA?

Your elevator pitch — practice saying this out loud in 30 seconds.



30-Second Pitch (memorise this)

"Auto EDA is a full-stack web application I built that automates exploratory data analysis. You upload any dataset — CSV, Excel, Parquet, Avro, or even pull directly from AWS S3 or Azure — and the platform instantly gives you statistical summaries, interactive visualizations, and a smart cleaning panel that detects issues and lets you selectively fix them. It also exposes a REST API so any downstream application can access your cleaned data with an API key. It's basically a no-code data exploration tool that handles everything from raw file ingestion to a publish-ready dataset."

The Problem It Solves

Every data project starts the same way: a data analyst spends hours writing the same pandas code — checking nulls, plotting distributions, removing duplicates, fixing column names. Auto EDA eliminates that repetitive work completely.



Before Auto EDA

Analyst writes Python scripts every time. 2–3 hours to understand a new dataset. No standard format. Hard to share results.



After Auto EDA

Upload and get full analysis in seconds. Share via REST API. Reproducible cleaning pipeline. Works for any file format.



Who uses it

Data analysts, data scientists, ML engineers, business analysts — anyone who needs to quickly understand a dataset.



What I built

Full-stack: React frontend + FastAPI backend + Pandas/PySpark processing + 4-provider cloud import + Docker deployment.

Key Numbers

METRIC	VALUE
Supported file formats	CSV, JSON, Excel (.xlsx/.xls), TXT, Parquet, Avro, Jupyter Notebook (.ipynb)
Cloud providers	Azure Blob Storage, AWS S3, GCP Cloud Storage, Databricks DBFS
Max upload size	500 MB (configurable)
Processing engines	Pandas (<100 MB) · PySpark (≥100 MB, auto-switched)
Chart types	Histogram, Box Plot, Bar Chart, Pie Chart, Scatter Plot, Correlation Heatmap
API routes	~25 REST endpoints across 9 route modules
Frontend pages	4 (Home, Dashboard, Multi-Dashboard, Image Dashboard)

SECTION 2

Features Deep Dive

What each feature does and how it works technically.

1. File Ingestion

Files are uploaded via drag-and-drop (react-dropzone). The backend receives the multipart form data, saves the file to `uploads/` with a UUID filename, and detects the format by file extension.

FORMAT	LIBRARY USED	NOTES
CSV / TXT	<code>pandas.read_csv()</code>	Auto-detects delimiter
JSON	<code>pandas.read_json()</code>	Handles nested JSON
Excel (.xlsx/.xls)	<code>openpyxl</code> / <code>xlrd</code>	Reads first sheet by default
Parquet	<code>pyarrow</code> / <code>fastparquet</code>	Column-oriented, very fast
Avro	<code>fastavro</code>	Reads schema + records
Jupyter (.ipynb)	<code>lxml</code> + <code>pandas</code>	Extracts DataFrame outputs from cell outputs

2. Statistical Analysis

The analysis service runs on the loaded DataFrame and computes:

```
shape          → row count, column count
dtypes         → data type per column (int, float, object, datetime)
missing        → count and % of nulls per column
duplicates     → total duplicate rows
describe()     → mean, std, min, 25%, 50%, 75%, max for numeric columns
corr()         → Pearson correlation matrix
IQR outliers   → values outside Q1 - 1.5×IQR or Q3 + 1.5×IQR per column
```

3. Selective Cleaning (Key Feature)

Unlike "clean everything" tools, Auto EDA **scans first, then asks**. The cleaning flow has two steps:



Operation IDs returned by the scan:

```
remove_duplicates      # if duplicates found
fill_missing__col_name # one entry per column with nulls
standardize_columns    # rename to snake_case
convert_numeric__col_name # if column looks numeric but stored as text
drop_outliers__col_name # IQR-based removal per column
```

Each operation has a **severity**: ● error (>10% affected), ● warning (moderate), ● info (minor).

4. Visualizations

Charts are generated server-side from the data and returned as JSON configs. The frontend renders them with **Plotly.js**. Each chart has a PNG download button that uses `Plotly.downloadImage()` at 2× retina resolution.

5. Multi-File Folder Upload

Two modes:

MODE	HOW IT WORKS
Separate	Each file gets its own dashboard tab. Files are processed independently.
Combined	Schema normalization → fuzzy deduplication (rapidFuzz) → merge into one DataFrame. Routed to the normal single-file Dashboard.

6. Cloud Import (Single File + Full Folder)

Users paste a URI and select a provider. The backend downloads the file(s) using the corresponding SDK (credentials from `.env`), saves to `uploads/`, and routes exactly like a local upload. Folder imports list all blobs/objects under a prefix and download all of them.

7. Data API with API Key Auth

After cleaning, users can generate a per-dataset API key. The key is shown once, then only its SHA-256 hash is stored. External apps can then call:

```
GET /api/data/{file_id}?page=1&page_size=100  
Header: X-API-Key: sk_abc123...
```

8. Image Dataset Analysis

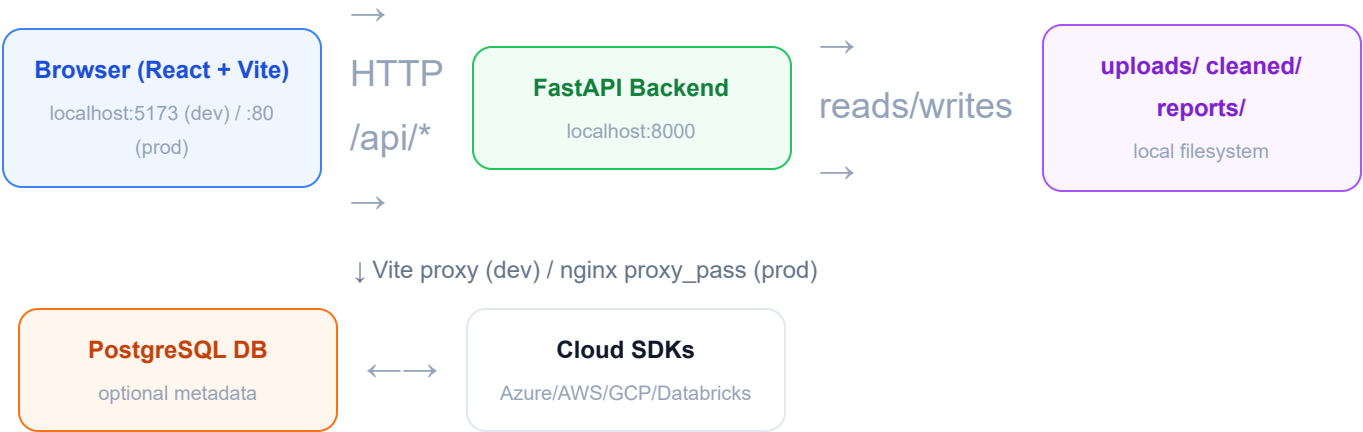
Upload a folder of images (PNG/JPG/BMP/WebP). The backend uses Pillow to detect: image count per class (folder), resolution stats, corrupted files, and duplicate images (via pixel hash). Outputs a clean ZIP download of valid images.

SECTION 3

Architecture & Tech Stack

System design and why each technology was chosen.

System Architecture



Why These Technologies?

FRONTEND React 18 + Vite React for component-based UI. Vite for blazing-fast HMR and build. Much faster than CRA.	STYLING Tailwind CSS Utility-first — no CSS files to maintain. Responsive breakpoints built in. Consistent design system.	CHARTS Plotly.js Interactive, supports 40+ chart types, has built-in PNG download, works great with React.
BACKEND FastAPI Async Python, automatic OpenAPI/Swagger docs, Pydantic validation, much faster than Flask.	DATA PROCESSING Pandas + PySpark Pandas for files under 100 MB. PySpark for large files — auto-switched based on file size.	DATABASE SQLAlchemy + PostgreSQL ORM for clean code. PostgreSQL for production. Graceful no-op if DATABASE_URL is not set.
DEPLOYMENT Docker + nginx Docker ensures consistent environment on any machine. nginx	FUZZY MATCH rapidFuzz Fast C++ implementation of Levenshtein/token ratio for fuzzy deduplication of combined datasets.	IMAGE Pillow (PIL) Standard Python image library. Used for reading image metadata,

serves static React build and proxies API calls.

detecting corruptions, computing pixel hashes.

Request Lifecycle (Single File Upload)

1. User drags CSV onto React dropzone
2. `POST /api/upload` → multipart/form-data → FastAPI
3. Backend saves file as `uploads/{uuid}.csv`
4. Returns `file_id` in JSON response
5. React navigates to `/dashboard/{file_id}`
6. Dashboard fetches `GET /api/analysis/{file_id}`
7. Backend loads file → runs Pandas analysis → returns JSON
8. React renders stats, chart configs returned by `GET /api/visualize/{file_id}`
9. User clicks Cleaning tab → `GET /api/clean/{file_id}/options`
10. User selects ops → `POST /api/clean/{file_id}` with `{"selected": ["remove_duplicates", ...]}`
11. Cleaned file saved to `cleaned/{file_id}.csv`
12. User downloads via `GET /api/download/{file_id}/cleaned`

SECTION 4

API Design & Security

REST endpoints, authentication, and security considerations.

Complete API Endpoint Map

METHOD	ENDPOINT	WHAT IT DOES
POST	/api/upload	Upload a single file
POST	/api/multi/upload?mode=	Upload a folder (separate or combined mode)
POST	/api/image/upload	Upload an image folder
POST	/api/cloud/import	Import from Azure/AWS/GCP/Databricks
GET	/api/analysis/{file_id}	Get statistical analysis
GET	/api/visualize/{file_id}	Get chart data (Plotly configs)
GET	/api/clean/{file_id}/options	Get detected cleaning issues
POST	/api/clean/{file_id}	Apply selected cleaning operations
GET	/api/download/{file_id}/cleaned	Download cleaned CSV
GET	/api/download/{file_id}/report	Download report JSON
GET	/api/data/{file_id}	Paginated data access (requires X-API-Key)
GET	/api/data/{file_id}/schema	Column names and types
POST	/api/key/{file_id}/regenerate	Rotate the API key

API Key Security

When a user clicks "Generate API Key":

```
# 1. Generate a cryptographically random key
raw_key = secrets.token_urlsafe(32) # e.g. "sk_abc123..."

# 2. Hash it before storing
key_hash = hashlib.sha256(raw_key.encode()).hexdigest()

# 3. Store only the hash (in uploads/.keys/{file_id}.key)
# 4. Return the raw key to the user ONCE — never stored in plain text

# On each API request:
incoming_hash = hashlib.sha256(x_api_key.encode()).hexdigest()
if incoming_hash != stored_hash:
    raise HTTPException(401, "Invalid API key")
```



Why SHA-256 hashing?

Even if someone gains access to the server filesystem, they only see hashes — they cannot recover the original API keys. This follows the same pattern used by GitHub and npm for their tokens.

CORS Configuration

Cross-Origin Resource Sharing is configured in `main.py` using FastAPI's `CORSMiddleware`. Origins are read from `ALLOWED_ORIGINS` in `.env` — so in production you add only your actual frontend domain, not `*`.

Input Validation

- All request bodies are validated with **Pydantic models** — invalid data returns 422 Unprocessable Entity automatically
- File extension is checked before processing — only known formats accepted
- File size is checked against `MAX_UPLOAD_SIZE_MB` before saving
- Cloud URIs are parsed and validated before any SDK call is made
- UUID file IDs prevent path traversal attacks — files are always accessed as `uploads/{uuid}.ext`

SECTION 5

Interview Questions & Answers

30+ likely follow-up questions with honest, easy-to-understand answers.

About the Project

Q1 Why did you build this project?

I wanted to build something end-to-end that combines data engineering with web development. Every data scientist I know wastes hours writing the same "first look at data" code — nulls, dtypes, distributions. I wanted to automate that completely, and also learn FastAPI, React, and Docker in a real project rather than a tutorial.

Q2 Walk me through what happens when a user uploads a file.

1. User drags a file onto the React dropzone component
2. Frontend sends a `POST /api/upload` request with the file as multipart form data
3. FastAPI saves the file to `uploads/` with a UUID as filename to avoid conflicts
4. The upload route returns a `file_id` and some metadata
5. React immediately navigates to `/dashboard/{file_id}`
6. The dashboard makes a `GET /api/analysis/{file_id}` call to get stats
7. The backend loads the file with the appropriate pandas reader and computes all statistics
8. Results are returned as JSON and rendered in the UI

Q3 What is the difference between the "Separate" and "Combined" folder modes?

Separate mode: Every file is processed independently and gets its own tab in the Multi-Dashboard. Good when the files are completely different datasets.

Combined mode: The backend tries to merge all files into one dataset. First it normalizes column names (lowercase, snake_case), then uses fuzzy string matching with rapidFuzz to find columns that are the same concept but spelled differently (like "customer_name" vs "CustomerName"), then concatenates everything. Good when you have monthly files of the same data structure.

Q4 How does the selective cleaning work?

There are two API calls involved. First, `GET /api/clean/{file_id}/options` scans the dataset and returns a list of detected issues — each issue has an ID, a label, and a severity level (error/warning/info). The frontend shows these as a checklist. The user checks only what they want to fix, then clicks Apply. The second call `POST /api/clean/{file_id}` receives the selected operation IDs in the request body and applies only those operations. So if you only want to remove duplicates and fill missing values in one column, you can do exactly that.

SECTION 5 (CONTINUED)

Interview Questions & Answers

Technical deep-dive questions.

Technical Questions

Q5 Why FastAPI instead of Flask or Django?

Three main reasons: **(1) Speed** — FastAPI is one of the fastest Python frameworks, comparable to Node.js, because it's built on Starlette which uses async I/O. **(2) Automatic docs** — it generates a Swagger UI at `/docs` automatically from my code, which is great for development and sharing. **(3) Pydantic** — all request/response bodies are validated automatically with type hints, so I get input validation for free without writing extra code.

Q6 Why do you use PySpark for large files? Couldn't Pandas handle it?

Pandas loads the entire file into RAM as a DataFrame. A 500 MB CSV might expand to 2–3 GB in memory because of Python's object overhead. On a machine with 8 GB RAM, that can cause an OOM (out of memory) crash. PySpark processes data in chunks (partitions) and can spill to disk when needed, so it can handle files that are much larger than available RAM. I auto-switch at 100 MB — below that Pandas is faster because there's no JVM startup overhead.

Q7 How do you handle file storage? Did you consider a database for files?

Files are stored on the local filesystem in `uploads/`, `cleaned/`, and `reports/` directories. I intentionally avoided storing binary data in a database because it's much slower and makes the DB grow huge. The UUID filename acts as the primary key. For production at scale, the right upgrade would be to store files in object storage like AWS S3 or Azure Blob and just store the URL in the database — that's actually what Docker volumes enable even now, by mounting those folders as volumes outside the container.

Q8 How does the API key authentication work?

When a user generates an API key, I use Python's `secrets.token_urlsafe(32)` to generate a cryptographically random key. I then compute its SHA-256 hash and store **only the hash** to disk. The raw key is shown to the user once and never stored. When an API request comes in with the key in the `X-API-Key` header, I hash the incoming key and compare it to the stored hash. This means even if an attacker reads the server's file system, they can't recover any API keys.

Q9 Why does the Plotly import use `plotly.js/dist/plotly` instead of just `plotly.js` ?

This was an actual bug I hit during development. `import Plotly from 'plotly.js'` imports the raw source, which has a dependency on Node.js's `buffer` module that doesn't exist in the browser. Vite's bundler throws a "Cannot resolve 'buffer/'" error. The fix is to import from `plotly.js/dist/plotly` — the pre-bundled browser build that has no Node.js dependencies.

Q10 How does cloud import work under the hood?

The user pastes a URI (like `s3://my-bucket/data/sales.csv`) and selects the provider. The React component sends a POST to `/api/cloud/import` with the provider name, URI, and import type (file or folder). The backend parses the URI to extract bucket and key/prefix, then calls the appropriate SDK — boto3 for AWS, azure-storage-blob for Azure, google-cloud-storage for GCP, databricks-sdk for Databricks. The file is downloaded to the local `uploads/` folder with a new UUID name and then processed exactly like a local upload. Credentials are read from `.env` and never exposed to the frontend.

Q11 Why Docker? What does the docker-compose.yml set up?

Docker solves the "works on my machine" problem. The docker-compose.yml defines two services: the **backend** (Python 3.11 slim + Java for PySpark, runs uvicorn on port 8000) and the **frontend** (a multi-stage build: Node builds the React app, then nginx serves the static files on port 80 and proxies `/api/*` requests to the backend). Volumes are mounted for `uploads/`, `cleaned/`, and `reports/` so data persists even if the containers restart.

Q12 What is CORS and why is it needed here?

CORS (Cross-Origin Resource Sharing) is a browser security feature. When the React app at `localhost:5173` tries to call the FastAPI backend at `localhost:8000`, the browser blocks it because they're on different ports (different "origins"). CORS middleware on the backend tells the browser "yes,

requests from localhost:5173 are allowed." In production I would restrict `ALLOWED_ORIGINS` to only the actual frontend domain — never use `*` (allow all) in production.

Q13 How does the multi-stage Docker build for the frontend work?

Stage 1 (build): Uses a Node.js image, copies the source, runs `npm ci && npm run build`. This produces a `dist/` folder with just HTML, CSS, and JS — no Node.js runtime needed. Stage 2 (serve): Uses a lightweight nginx image, copies only the `dist/` folder from Stage 1. The final image is tiny (≈25 MB) because it doesn't include Node.js or node_modules. nginx also has a config that proxies `/api/` to the backend container.

SECTION 5 (CONTINUED)

Interview Questions & Answers

Data, design, and behavioral questions.

Data & Design Questions

Q14 How do you detect outliers?

I use the **IQR (Interquartile Range) method**. For each numeric column: calculate Q1 (25th percentile) and Q3 (75th percentile), then $IQR = Q3 - Q1$. Any value below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ is flagged as an outlier. This method is robust because it's based on percentiles and not affected by the outliers themselves. I chose it over Z-score because Z-score assumes a normal distribution, while IQR works for any distribution.

Q15 How do you fill missing values?

The strategy depends on the column type. For **numeric columns**, I fill with the column median (more robust than mean because it's not skewed by outliers). For **categorical/text columns**, I fill with the string "Unknown". This is exposed as a cleaning option per column — the user can choose to apply it or skip it for any specific column.

Q16 What would you do differently if you had to scale this to 100,000 users?

Several things: **(1) File storage** — move to S3/Azure Blob instead of local disk. **(2) Processing** — use a task queue like Celery + Redis so large file processing happens in the background and doesn't block HTTP requests. **(3) Database** — move from optional PostgreSQL to a required one for all metadata. **(4) Caching** — cache analysis results in Redis since the same file's stats don't change. **(5) Horizontal scaling** — the backend would need to be stateless (no local files) to run multiple instances behind a load balancer.

Q17 How does fuzzy deduplication work in combined mode?

When merging multiple files, column names might be slightly different — "CustomerName", "customer_name", "cust_name" all mean the same thing. I use **rapidFuzz** (a fast implementation of fuzzy

string matching) to compute similarity ratios between column names across files. If two column names have a similarity above a threshold (e.g., 85%), they're treated as the same column. This handles real-world messy data where different people named columns slightly differently.

Q18 What is the Database used for? The app seems to work without it.

Exactly — the database is optional and stores only **metadata** (not the files themselves): file ID, original filename, upload source, file format, file size, and upload timestamp. This lets you build features like an upload history page or search past uploads. Without it, the app is fully functional but stateless — if you restart the server, you lose track of which file IDs exist (though the files themselves are still on disk).

Behavioral Questions

Q19 What was the hardest bug you fixed?

The Plotly build error was tricky. The frontend built fine in development but failed with "Cannot resolve 'buffer/'" when I ran `npm run build`. After debugging I traced it to the Plotly import — `import Plotly from 'plotly.js'` imports the source code which has a Node.js dependency. The fix was switching to `plotly.js/dist/plotly` (the pre-built browser bundle). I learned that for browser-targeted libraries you always want the `dist` build, not the source.

Q20 What would you add next to improve this project?

A few things I'd prioritize: **(1) Async processing** — use Celery so large files don't timeout the HTTP request. **(2) User accounts** — OAuth login so users can access their upload history. **(3) AI suggestions** — use an LLM to describe what patterns it sees in the data in plain English. **(4) Data versioning** — track cleaning history so you can undo/redo. **(5) Export to more formats** — not just cleaned CSV, but also Parquet or directly push to a database table.

Q21 How did you make the UI responsive?

I used Tailwind CSS's responsive prefixes. The home page is a two-column layout on desktop (`lg:grid-cols-2`) and single-column on mobile. The multi-file dashboard has a sidebar on desktop and a horizontal scroll file picker on mobile (hidden with `hidden lg:block`). All dashboards use `max-w-screen-2xl` to fill wide monitors. The tab bars use `overflow-x-auto` so they scroll horizontally on small screens instead of wrapping badly.

Q22 How would you add user authentication to this project?

I'd add a `/auth` route with `POST /auth/register` and `POST /auth/login`. Login would return a JWT (JSON Web Token) signed with a secret key. The frontend stores the JWT in memory (or localStorage) and sends it as an `Authorization: Bearer {token}` header on every request. FastAPI's `Depends` system would extract and verify the JWT on each protected route. Each uploaded file would be linked to a user ID so users only see their own data.

SECTION 6

Challenges & How I Solved Them

Real problems faced during development — perfect for "tell me about a challenge" behavioral questions.

CHALLENGE	WHAT HAPPENED	HOW I FIXED IT
Plotly build error	Vite threw "Cannot resolve 'buffer/' on production build even though dev worked	Traced to wrong Plotly import path. Changed <code>'plotly.js'</code> → <code>'plotly.js/dist/plotly'</code>
PNG download button broken	<code>window.Plotly</code> was undefined in module code — <code>downloadImage()</code> failed silently	Imported Plotly directly: <code>import Plotly from 'plotly.js/dist/plotly'</code> and called <code>Plotly.downloadImage()</code>
CleaningPanel syntax error	"return outside of function" — leftover code from a previous version was still in the file after a partial edit	Read the full file, found the dangling code block after the new component, removed it
Uvicorn module not found	Running <code>uvicorn main:app</code> from project root failed because Python couldn't find <code>main.py</code>	Must always run from inside <code>backend/</code> directory. Documented this clearly.
Missing closing div in DownloadPanel	After a replace operation, the API key card was missing its opening <code><div</code> tag, breaking the JSX parse	Carefully re-read the component structure and restored the missing tag
Large file memory crash	A 300 MB CSV caused the backend	Implemented the PySpark fallback — files ≥ 100 MB are auto-routed to PySpark which processes data in partitions

CHALLENGE	WHAT HAPPENED	HOW I FIXED IT
	to run out of memory and crash	
CORS in production Docker	API calls failed in Docker because ALLOWED_ORIGINS still listed <code>localhost:5173</code> (dev URL)	Updated docker-compose.yml to pass <code>ALLOWED_ORIGINS=http://localhost,http://localhost:80</code>
Avro reading	Standard Pandas can't read Avro files natively	Added <code>fastavro</code> library — reads schema and records, converts to DataFrame

**Tip for behavioral answers**

When asked "tell me about a challenge," use the **STAR format**: Situation (what was happening), Task (what you needed to fix), Action (exactly what you did), Result (what improved). Pick the Plotly build error or the memory crash — they have the clearest cause → fix → learning arc.

SECTION 7

Resume Bullet Points

Copy these into your CV. Each bullet follows the **Action** → **Technology** → **Impact** format that ATS systems and recruiters respond to best.

PROJECT TITLE LINE

- ▶ **Auto EDA** — Automated Exploratory Data Analysis Platform | Python · React · Docker · Cloud | github.com/ChaudharyPratham/auto_eda

BACKEND & API

- ▶ **Built a full-stack data analysis platform** using FastAPI and React 18, enabling no-code exploratory data analysis for CSV, JSON, Excel, Parquet, Avro, and Jupyter Notebook formats
- ▶ **Designed and implemented 25+ REST API endpoints** across 9 route modules (upload, analysis, cleaning, visualization, download, multi-file, image, cloud, data API) with Pydantic request validation and Swagger auto-documentation
- ▶ **Implemented per-dataset API key authentication** using SHA-256 hashing (Python secrets module) — keys stored as hashes, never in plain text, following industry security best practices
- ▶ **Integrated auto-scaling processing pipeline** that routes files under 100 MB to Pandas and files ≥ 100 MB to PySpark local mode, preventing OOM crashes on large datasets

DATA ENGINEERING

- ▶ **Built selective data cleaning pipeline** that scans datasets for issues (duplicates, nulls, type mismatches, outliers using IQR method) and presents a severity-ranked checklist — users apply only chosen operations
- ▶ **Implemented multi-file schema normalization and fuzzy deduplication** using rapidFuzz (Levenshtein/token ratio) to merge datasets with inconsistent column naming conventions
- ▶ **Integrated 4-provider cloud import** (Azure Blob Storage, AWS S3, GCP Cloud Storage, Databricks DBFS) supporting both single-file and full folder/prefix downloads, with credentials isolated to server-side .env

FRONTEND & VISUALIZATION

▶

Developed responsive React frontend

with 4 pages and 10+ reusable components, using Tailwind CSS responsive grid (mobile-first) and react-router-dom v6 for SPA navigation

▶

Built interactive visualization dashboard

with 6 Plotly.js chart types (histogram, box plot, bar, pie, scatter, heatmap) including 2× retina-quality PNG export via Plotly.downloadImage()

DEVOPS & DEPLOYMENT

▶

Containerised full-stack application

using Docker Compose with multi-stage frontend build (Node → nginx) and Python 3.11 backend with Java/PySpark support — deployable with single `docker compose up --build` command

▶

Configured nginx reverse proxy

to serve React static build on port 80 and forward /api/* requests to FastAPI backend, with SQLAlchemy + PostgreSQL metadata storage (optional, graceful fallback)

Skills to List on Resume (from this project)

CATEGORY	TECHNOLOGIES / SKILLS
Languages	Python 3.11, JavaScript (ES2022), SQL
Backend	FastAPI, Uvicorn, Pydantic, SQLAlchemy, REST API design
Data	Pandas, NumPy, PySpark, Parquet, Avro, ETL, Data Cleaning, EDA
Frontend	React 18, Vite, Tailwind CSS, Plotly.js, react-router-dom
Cloud	Azure Blob Storage, AWS S3 (boto3), GCP Cloud Storage, Databricks
DevOps	Docker, Docker Compose, nginx, multi-stage builds
Security	SHA-256 hashing, CORS configuration, API key auth, input validation
Database	PostgreSQL, SQLAlchemy ORM
Tools	Git, Swagger/OpenAPI, Postman, VS Code

**ATS tip**

Include the words "FastAPI", "Docker", "PySpark", "REST API", "AWS S3", "React" explicitly in your resume — these are the exact keywords that Applicant Tracking Systems scan for in data engineering and full-stack roles.